

UCVM Staging Environment Guide

Updated: October 6, 2026
Authors: Jasneet Singh & Eze Adiele

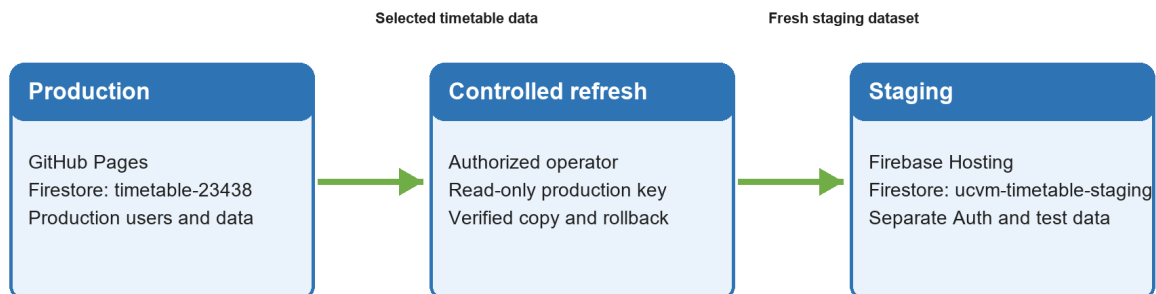
Purpose and scope

The UCVM staging environment is a separate, safe copy of the timetable system used to test changes before they affect the live timetable. It has its own website, Firestore database, authentication accounts, and operational credentials. Production remains independent.

Production	Staging
GitHub Pages site and Firebase project timetable-23438	Firebase Hosting site and Firebase project ucvm-timetable-staging
Live timetable and live user activity	Testing data and selected staging testers
Never written by the staging refresh tool	Can be refreshed from selected production timetable data

How the environment works

Staging is intentionally isolated. A page served from Firebase Hosting loads the staging configuration, connects only to the staging Firebase project, and keeps its browser cache separate from production.



The refresh process reads only selected timetable data from production and writes only to staging. Production users, authentication records, change requests, access requests, and audit logs are not copied.

What is refreshed

Included	Not copied from production
sessions	Firebase Authentication users and passwords

Included	Not copied from production
roster	users and authorized user profiles
settings/rosterNotice	change requests, access requests, history, change logs, audit logs
A new staging-only sessionsVersion timestamp	Production sessionsVersion timestamp

During a refresh, staging displays a short maintenance state. After the copied data is verified, the staging sessionsVersion timestamp is updated once. This tells open staging browsers to replace any older cached timetable data with the refreshed staging data.

Where to find each component

Component	Where to find it	Why it matters
Staging website	https://ucvm-timetable-staging.web.app	Safe place to test the deployed application.
Source code	Timetable repository, staging-branch	Contains the staging application and operations tools.
Hosting settings	firebase.json and .firebaserc	Targets the staging Firebase project and excludes the Excel data feed.
Firestore rules	firestore.rules	Controls browser access and blocks browser writes during a refresh.
Refresh tool	scripts/refresh-staging.mjs	Copies approved data, verifies it, and rolls back if necessary.
Tester allowlist	scripts/seed-staging-allowlist.mjs	Adds selected testers without copying production accounts.
Operations guide	docs/staging-environment-runbook.md	Exact setup, deployment, refresh, and recovery commands.

Firebase Hosting and Google Cloud access

Firebase Hosting publishes the staging branch to the staging site. The site configuration deliberately does not publish data-feed.html, documentation, scripts, dependencies, or credential files.

Two purpose-limited Google Cloud service accounts support a refresh. Their exact email addresses and JSON keys are not kept in the repository. Find and manage them in Google Cloud Console by selecting the appropriate project, then opening IAM & Admin > Service Accounts.

Access role	Project	Permitted use
Production reader	timetable-23438	Read Firestore data required for the approved staging copy. It cannot write production data.
Staging writer	ucvm-timetable-staging	Write, verify, and if needed roll back staging data. It cannot access production.

Keys must be stored outside the repository and outside OneDrive. The refresh tool checks both project IDs before it begins. Browser users never receive these keys.

Access and day to day operation

- Staging users may use the same email address as production, but they need a separate staging password because Firebase Authentication is a separate user pool.
- A staging administrator creates selected tester entries in the staging allowlist. Users then create or reset their own staging credentials through the site.
- Before a refresh, run the read-only dry run. Apply a refresh only from a trusted operator computer with both service-account paths set for that PowerShell session.
- If a refresh fails, the tool restores the previous staging snapshot and leaves the cache timestamp unchanged. If rollback fails, staging stays in maintenance until an authorized operator resolves it.

Common questions

Will staging change the live timetable?

No. The refresh tool reads production but never writes to it. Staging and production use different Firebase projects.

Why do testers need a separate password?

Authentication is intentionally separate. This keeps staging tests, user IDs, and password resets away from production.

Why can a refresh affect an open browser?

Browsers cache timetable data for speed. The final staging-only version update tells them to fetch the refreshed data.

Can the staging Excel data feed be used?

No. data-feed.html is intentionally excluded from Firebase Hosting to avoid an unnecessary export endpoint and extra reads in staging.

Where are credentials stored?

Service-account keys are held by authorized operators outside the repository. Passwords and JSON keys should never be added to source control or shared in this guide.

Who should run a refresh?

Only a trusted technical operator with the two purpose-limited Google Cloud service-account keys and the staging runbook.